



THE FREE GUIDE

B2B Software & IT Buyer's Guide

How to choose, scope, and budget custom software
and IT services - without the expensive mistakes.

INSIDE THIS GUIDE

- | | |
|-------------------------------------|--------------------------------------|
| 01 Build vs. buy vs. customise | 04 Red flags that predict failure |
| 02 The real cost of custom software | 05 Owning your software (no lock-in) |
| 03 10 questions to ask any partner | 06 An 8-step scoping checklist |

INTRODUCTION

Buy software like it's an investment, not a purchase

Most businesses don't fail at software because the technology is hard. They fail because the **decisions around it** are made without a clear framework - the wrong build-vs-buy call, a budget set by guesswork, or a partner chosen on price alone. This short guide gives you that framework: practical, jargon-free, and written from real B2B delivery experience.

1 Build vs. buy vs. customise

Three paths, and most teams pick the wrong one by default. Use this rule of thumb:

- **Buy off-the-shelf** when the process is generic and you're happy to adapt to the tool (e.g. email, accounting).
- **Customise a foundation** when 70% of what you need exists but the last 30% is your competitive edge.
- **Build custom** when the workflow *is* the business - the thing no off-the-shelf tool models well.

The real question

Don't ask "what software should we buy?" Ask "where does our process differ from everyone else's?" Buy the generic parts; build only the parts that make you different.

2 The real cost of custom software

The build is rarely the biggest cost - the **total cost of ownership** is. Budget for all five:

- **Discovery & design** - scoping the right thing before writing code (saves the most money).
- **Build** - the visible cost everyone quotes.
- **Integration & data** - connecting existing tools and migrating data is often underestimated.
- **Hosting & infrastructure** - cloud, backups, monitoring; ongoing, not one-off.
- **Maintenance & support** - budget 15-25% of build cost per year to keep it healthy.

Rule of thumb

If a quote only covers "the build," it's incomplete. A trustworthy partner shows you the ownership cost up front - even the parts that aren't theirs to bill.

3 10 questions to ask any software or IT partner

- Who owns the code and the IP when we're done?
- What does handover include - documentation, repo access, deployment?
- How do you scope and price: fixed bid, sprints, or time-and-materials?
- What happens if we want to leave or switch teams mid-project?
- Who maintains it after launch, and what does support cost?
- How do you handle security, access, and our data?
- Can we see a project you delivered in our industry or stack?
- Who is actually on the team - seniors, or juniors behind a sales pitch?
- How will we see progress (demos, standups, environments)?
- What's your plan for the parts you *don't* build (hosting, integrations)?

Green flag

A good partner answers "who owns the code?" with "you do" - instantly, in writing. Anything vaguer is a lock-in risk.

4 Red flags that predict a failed project

- A fixed price quoted before anyone understands the workflow.
- No discovery phase - straight to "we'll start building Monday."
- Vague ownership of code, accounts, or data.
- One salesperson, then silence - you never meet the builders.
- No staging environment or demos; you only see the end result.
- Pressure to sign fast with a discount that "expires Friday."

5 Owning your software: handover & no lock-in

You should be able to walk away with everything and keep running. Insist on:

- Source code in *your* repository, not the vendor's private one.
- Your own cloud, domain, and service accounts - in your name.
- Documentation a new developer could pick up cold.
- A clean deployment process (CI/CD), not a black box only they can run.

6 Scope your first project in 8 steps

- ☐ 1. Write the problem in one sentence - the outcome, not the feature.
- ☐ 2. List the workflow steps that are manual or error-prone today.
- ☐ 3. Mark which steps are generic (buy) vs. your edge (build).
- ☐ 4. Name the systems it must connect to (CRM, ERP, payments, email).
- ☐ 5. Decide who owns code, cloud, and data (answer: you).
- ☐ 6. Set a budget range including ownership cost, not just build.
- ☐ 7. Agree how you'll see progress - demos and environments.
- ☐ 8. Define "done": handover, docs, deployment, and support terms.

+ How Musk-IT can help

Musk-IT is a B2B custom software and IT solutions provider. We build software around your real workflows - ERP, CRM, dashboards, web and mobile apps, automation - and run the IT behind it: cloud & infrastructure, managed IT & support, cybersecurity, and IT consulting. You own the code, with no lock-in.

Two free ways to start

1) Book a free consultation and we'll help you apply this guide to your project. 2) Claim a free IT & security audit - we'll review your cloud, security, and systems and hand you a prioritised action plan. Visit muskit.in to get started.

muskit.in - Custom Software & IT Solutions - (c) 2026 Musk-IT. This guide is general information, not professional advice.